Loss function

In deep learning, a **loss function** is a mathematical function that measures the difference between the predicted output of a model and the actual target values.

Types of Loss Functions:

Loss functions vary based on the type of problem: regression, classification, or other specialized tasks.

1. Regression Loss Functions

Used when the target variable is continuous.

- **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

  - Penalizes larger errors more severely.
  - Suitable for predicting continuous values.

- **Mean Absolute Error (MAE)**:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

  - Less sensitive to outliers than MSE.

- **Huber Loss**: Combines MSE and MAE, robust to outliers.

2. Classification Loss Functions

Used when the target variable is categorical.

- **Binary Cross-Entropy (Log Loss)**:

$$\text{Loss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

  - Used for binary classification problems.

- Measures the dissimilarity between predicted probabilities and actual labels.

- **Categorical Cross-Entropy**:

$$\text{Loss} = -\sum_{i=1}^{n}\sum_{j=1}^{k} y_{ij}\log(\hat{y}_{ij})$$

  - Generalization of binary cross-entropy for multi-class classification.

Key Points:

Relu, Softmax, CategoricalCrossEntropy => multi-class

Relu, Sigmoid, BinaryCrossEntropy => Binary class

Relu, Linear Activation, {MSE, MAE, Huber Loss} =>Linear Regression

Optimizers

Optimizers are algorithms or methods used to minimize the loss function by updating the weights and biases of a neural network during training.

Role of Optimizers

1. **Minimize Loss**:
   - Optimizers adjust the parameters (weights and biases) to minimize the difference between predicted and actual values (i.e., minimize the loss function).
2. **Improve Convergence**:
   - They help the model converge to an optimal or near-optimal solution efficiently.
3. **Handle High-Dimensional Spaces**:
   - Optimizers navigate the complex, high-dimensional loss landscape.

1. Gradient Descent (GD)

- **Description**:
  - Updates the parameters in the opposite direction of the gradient of the loss function.

$$\theta = \theta - \eta \cdot \nabla J(\theta)$$

  - θ: Model parameters (weights).
  - η: Learning rate.
  - $\nabla J(\theta)$: Gradient of the loss function.
- **Variants**:
  - **Batch Gradient Descent**:
    - Uses the entire dataset to compute gradients.
    - Slow for large datasets.
  - **Stochastic Gradient Descent (SGD)**:
    - Updates weights using a single data point at a time.
    - Faster but noisier.
    - Introduces noise, which can help escape local minima.
  - **Mini-Batch Gradient Descent**:
    - Updates weights using small batches of data.
    - Balances speed and stability.

Momentum

- Momentum enhances gradient descent by adding inertia, which accelerates convergence and smoothens oscillations.

$$v_t = \gamma v_{t-1} + \eta \nabla J(\theta)$$
$$\theta = \theta - v_t$$

  - vt: Velocity term.
  - γ: Momentum coefficient (e.g., 0.9).
  - η: Learning rate.

  Benefits:

- Speeds up convergence along gentle slopes.
- Reduces oscillations in steep directions.

 Drawbacks:

- May overshoot minima if momentum is too high.

1. Adaptive optimizers

- Adaptive optimizers are optimization algorithms that adjust the learning rate dynamically for each parameter during training.
- **Dynamic Learning Rate**: They adjust the learning rate for each parameter individually, making training faster and more stable.

1. Adagrad

- **Key Idea**:
  - Adapts learning rates based on the magnitude of past gradients.

- **Formula**:

$$\theta = \theta - \frac{\eta}{\sqrt{\sum g^2 + \epsilon}} \cdot g$$

  - g: Gradient (a partial derivative of the loss function with respect to the parameter).
  - Summation(g^2): summation all previous weight
  - ϵ: Small constant for numerical stability.

2. RMSprop

- **Key Idea**:
  - Improves upon Adagrad by using an exponentially decaying average of squared gradients instead of summing them.
- **Formula**:

$$\theta = \theta - \frac{\eta}{\sqrt{E[g^2] + \epsilon}} \cdot g$$

  - E[g2]: Exponentially weighted moving average of squared gradients.

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \cdot \hat{m}_t$$

3. Adam (Adaptive Moment Estimation) optimizer

The **Adam (Adaptive Moment Estimation)** optimizer combines concepts from **momentum** and **adaptive learning rates**.

Formulas:

- First Moment (Mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g$$

  - mt: The exponentially weighted average of past gradients, representing the "mean" or first moment of gradients.
  - β1: Decay rate for the moving average of gradients (0.9).
  - g: Current gradient (i.e., partial derivative of the loss function with respect to the parameter).

- Second Moment (Variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g^2$$

  - vt: The exponentially weighted average of past squared gradients, representing the "variance" or second moment.
  - β2: Decay rate for the moving average of squared gradients (typical value: 0.999).
  - g2: Square of the current gradient (element-wise).

- Bias Correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

  - β1t: Powers of β1 and β2, decreasing as t (iteration count) increases.
  - m^t: Bias-corrected first moment (mean).
  - v^t: Bias-corrected second moment (variance).

- Update Rule:

$$\frac{\partial L}{}$$

$$\theta = \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

$$\left[\theta_n = \theta_0 - \eta \frac{\partial}{\partial \theta}\right]$$

- θ: Model parameters (weights and biases).
- η: Learning rate.
- m^t: Bias-corrected first moment.
- v^t: Square root of the bias-corrected second moment.
- ϵ: Small constant added for numerical stability (eg,10^{-8})

- **Advantages**:
  - Fast convergence.
  - Works well for large datasets and deep networks.
- **Disadvantages**:
  - Can sometimes overfit or fail to generalize well.